

Support bootstrap for zero touch initial configuration in DLPoD Appliance

Status: Implemented (shipped on `develop`) | **Epic:** [ENG-1071732](#) | **Story:** [ENG-1069636](#) | **PR:** [service#107176](#)

Post-merge fixes: [ENG-1163522](#) (cert DN value quoting), [ENG-1184961](#) (ESXi userdata JSON-string + wizard-hang), [ENG-1185239](#) (surface no-ops instead of silent skip), [ENG-1188189](#) (systemd ordering-cycle fix).

i This document covers the **appliance-side** zero-touch configuration design. It has been **revised**: bootstrap is now applied by a dedicated headless systemd service (`nsbootstrap.service`), not by the `nsconfigwizard` TUI; cloud-init is locked down so it *never executes* userdata; and the raw `commands[]` escape hatch has been removed. The superseded in-wizard design is recorded in §8. The AWS CloudFormation / Terraform / userdata structure that *produces* the bootstrap payload is a separate design.

Post-merge hardening — ESXi `guestinfo.userdata` JSON-string unwrapping, always-run no-op reporting, and the systemd ordering-cycle fix — is folded into §3, §4 and §7.

- 1. Context & Motivation
- 2. Design Overview (current)
 - End-to-end flow
- 3. systemd Service Ordering
 - Ordering
- 4. nsbootstrap Service (headless flow)
 - 4.1 Key enabling finding — interactive commands are single-message driveable
- 5. nsconfigwizard as Observer
- 6. Progress & Status Reporting
- 7. Security Design
 - 7.1 Principle
 - 7.2 Layer 1 — cloud-init lockdown
 - 7.3 Layer 2 — delivery & least privilege
 - 7.4 Layer 3 — strict validation (fail closed)
 - 7.5 Defense in depth
- 8. bootstrap.json Schema
- 9. Superseded Design (for the record)
- 10. Deferred / Open Items
- 11. Testing

1. Context & Motivation

Today the `nsconfigwizard` TUI requires an operator at the console on first login: it waits for `nssetupdlpaas.service` to install the DLP pods, then presents an interactive **persona selection** screen and, on network failure, DNS/IP forms.

For cloud and automated deployments we want a **zero-touch (unattended) provisioning** path. The appliance is launched via an **AWS CloudFormation Template / Terraform**, and **cloud-init userdata** carries a structured bootstrap configuration. On first boot the appliance must apply that configuration (DNS, interface, system, upstream proxy, certificates, persona) and install the DLP pods without any operator interaction.

The nsshell backend (`clib`) is reachable over ZMQ via `ClibClient` — the same client used for the interactive network-failure flow — so command execution, the two-phase `save` , readiness probing, and error capture are all reused. Zero-touch adds no new IPC path.

2. Design Overview (current)

All headless bootstrap work lives in a **new standalone package** `nsbootstrap.ns` that ships a one-shot systemd unit `nsbootstrap.service` . This decouples provisioning from the interactive login shell and yields a deterministic, auditable boot-time step.

- **nsbootstrap.service** — headless: read + strictly validate the bootstrap config, apply it over `clib`, write the persona/platform file, redact secrets, record progress.
- **nsconfigwizard** — now a **pure observer**: it polls `nsbootstrap.service` and its progress file, shows progress/errors, and handles persona selection only in the non-zero-touch case. It no longer reads or applies bootstrap config.
- **cloud-init** — a **data carrier only**. It is locked down so it caches userdata to disk but executes nothing (§7).

End-to-end flow

```
1
2 launch (CFN/Terraform, userdata = bootstrap JSON)
3   -> cloud-init CACHES userdata to /var/lib/cloud/instance/user-data.txt (never executes
   it)
4   -> system reaches multi-user.target
5   -> nsbootstrap.service: stage userdata -> validate JSON+schema -> apply via clib -> write
   persona -> finalize
6   -> nssetupdlpaas.service: install DLP pods using the applied config
7   nsconfigwizard (login shell) observes both services and shows progress
8
```

3. systemd Service Ordering

A blocking `Type=oneshot` unit that is both `WantedBy=` and `After=multi-user.target` forms a systemd **ordering cycle**; systemd breaks it by **deleting the unit's start job** — nothing runs, with no log or journal entry (ENG-1188189). The fix is `DefaultDependencies=no` on `nsbootstrap.service`, `nssetupdlpaas.service` and `nsreconciledlpaas.service` (if any one keeps its default dependencies the cycle re-closes through it). The units order on `cloud-config.service` — the latest cloud-init unit that is safe to depend on; `cloud-final.service` is itself `After=multi-user.target` and would drag the unit straight back into the cycle.

Ordering

```
1
2 cloud-config.service (userdata cached) -> nsbootstrap.service -> nssetupdlpaas.service
3 > nsreconciledlpaas.service
```

Unit	Key directives	Notes
nsbootstrap.service	Type=oneshot , DefaultDependencies=no , After=cloud-config.service , After=multi-user.target , WantedBy=multi-user.target , StartLimitBurst=5	Runs on every boot — no Condition* gating ; it classifies and reports absent / empty / already-applied itself (§7.3). StartLimitBurst kept ≥ the wizard's retry budget.
nssetupdlpaas.service	DefaultDependencies=no , After=multi-user.target , After= + Wants=nsbootst rap.service , WantedBy=multi-user.target	Installs pods <i>after</i> config is applied. Soft dependency: still runs if nsbootstrap fails (manual / non-cloud installs); the wizard surfaces any error.

<code>nsreconciledlpaas.service</code>	<code>DefaultDependencies=no</code>	Carries the same <code>DefaultDependencies=no</code> so the three-unit ordering cycle cannot re-close through it (ENG-1188189).
--	-------------------------------------	---

⚠️ Why not `WantedBy=cloud-final.service` or `WantedBy=nsbootstrap.service` ?

Both were evaluated and rejected: they couple the critical pod-installer's scheduling to cloud-init, which would prevent pods from installing on non-cloud form factors (OVA/ISO) where `cloud-final` may not run. `WantedBy=multi-user.target` keeps `nssetupdipaas` scheduled on every boot and every form factor.

4. nsbootstrap Service (headless flow)

- Stage input** — a root-only `ExecStartPre` copies cloud-init's root-owned userdata cache into a restricted (`0640 nsadmin`) file. The main process then runs unprivileged as `nsadmin`.
- Read + validate** — parse strictly as JSON, unwrapping the VMware/ESXi `guestinfo.userdata` JSON-string from one further level (ENG-1184961); reject non-JSON and any unknown top-level key (fail closed, §7.4).
- Apply** — for each config section, generate the ordered nsshell commands and send them over one clib session, then a single two-phase `save`. Interactive fields (PEM certs, passwords, banners, SSH keys) are inlined in the first message via `mlines` / `password` (see §5).
- Persona** — write `/opt/ns/appliance/common/persona` (+ platform for dspm) so `nssetupdipaas` installs the right pods. Pod-dependent persona side-effects (supervisor restart, dspm scale) run later in the wizard, post-install.
- Finalize** — write a redacted `bootstrap.json.done` (secrets blanked) and remove the staged copy. A done-marker makes the run idempotent.
- Progress** — every step is written atomically to a progress file (§6).

⚠️ The service carries **no** `Condition*` **gating** and runs on every boot: it classifies the staged userdata as *absent* / *empty* / *present* / *already-applied* and **announces the outcome** (the three no-op cases exit 0 with an explicit log line), so a platform that failed to deliver userdata is diagnosable rather than a silent systemd skip (ENG-1185239).

4.1 Key enabling finding — interactive commands are single-message driveable

Several nsshell commands are normally *interactive* (the `@multiline` / `@promptpassword` decorators do a two-round-trip prompt). Those decorators are **stateless per request** (`if "mlines" not in self.request` / `if "password" not in self.request`), so a generator can inline the value in the first `do` message and skip the prompt entirely:

```
1
2 {"cmd":"do","mode":"config","line":"set dlpaas server-cert","mlines":"
3  <PEM>","user":"nsadmin"}
4 {"cmd":"do","mode":"config","line":"set management-plane upstream-proxy-server
   password","password":"<plaintext>","user":"nsadmin"}
```

Only the command `line` is ever surfaced to logs/progress; the inlined `mlines` / `password` never leave the send scope.

5. nsconfigwizard as Observer

The wizard no longer applies bootstrap config. It now:

- Detects whether a bootstrap phase is in play (`nsbootstrap.service` loaded + a progress file / done-marker / failure).
- Polls `nsbootstrap.service` and its progress file and renders a progress bar — reusing the same schema-v1 step renderer as the installer.
- On bootstrap failure, shows the failing command + error + log path + support guidance, then drops to nsshell.
- Then observes `nssetupdlpaas` (unchanged) and, in zero-touch, skips persona selection (already written by nsbootstrap) and applies post-install side-effects. Non-zero-touch keeps the classic interactive flow.

6. Progress & Status Reporting

nsbootstrap writes `/opt/ns/states/nsbootstrap/progress.json` using the DLP installer's **schema v1** (so one wizard renderer serves both), plus an `error` object. Writes are **atomic** (temp file + `os.replace`) so the polling wizard never reads a partial document.

```
1
2 {
3   "status": "in_progress | successful | failed",
4   "retry_count": 0,
5   "steps_schema_version": 1,
6   "steps": [ {"id": "...", "label": "...", "category": "...",
7               "status": "pending | active | completed | failed | skipped"} ],
8   "error": { "command": "<failing cmd, no secrets>", "message": "<clib error>" }
9 }
10
```

7. Security Design

✗ cloud-init has **no native script signing or allowlist**. Its `runcmd` / `bootcmd` / `user-scripts` run **as root at boot**. Left at defaults, userdata is an arbitrary-root-code-execution channel. The design therefore removes cloud-init's ability to execute anything and routes config through a single trusted, schema-validated processor.

7.1 Principle

cloud-init is a data carrier, not an executor. Two independent layers: (1) neutralize cloud-init's execution engine; (2) make nsbootstrap the sole authorized processor, which only accepts schema-valid JSON and only emits a closed set of clib-validated nsshell verbs.

7.2 Layer 1 — cloud-init lockdown

An image-baked drop-in `/etc/cloud/cloud.cfg.d/99-netskope-lockdown.cfg` (written by `dlp_modify_rootfs.sh`):

- **Allow the expected datasources** — `datasource_list: [ Ec2, Azure, GCE, OpenStack, ConfigDrive, VMware, OVF, NoCloud, None ]` to support AWS / Azure / GCP / OpenStack / VMware-ESXi / KVM / Hyper-V. Because NoCloud/ConfigDrive/OVF are seed-from-media transports, the datasource list is **not** the security boundary; safety comes from the disabled execution modules (below) and nsbootstrap strict validation (§7.4) — cloud-init executes nothing and only schema-valid config *values* are ever applied.
- **Disable vendor-data** — `vendor_data` / `vendor_data2` disabled (another script-execution channel).
- **Minimal module lists** — every executing / arbitrary-write / network-fetch / config-management / callback module removed: `bootcmd`, `write_files(_deferred)`, `runcmd`, `scripts-user` / `-vendor` / `-per-*`, `package-update-upgrade-install`, `snap`, `apt-configure`, `ssh-import-id`, `puppet` / `chef` / `ansible` / `salt-minion` / `mcollective`, `phone-home`, `power-state-change`, `users-groups`.

Residual: raw `#!` shell-script and `#cloud-boothook` userdata are handled by cloud-init part-handlers that run before the module list, so module-stripping alone does not cover them. Mitigated by (a) delivering bootstrap config as raw JSON — an inert userdata type — and (b) IAM restricting who can set instance `userData`.

7.3 Layer 2 — delivery & least privilege

The bootstrap payload **is** the raw userdata; there is no cloud-init `write_files` step (which would be an arbitrary-file-write hole). cloud-init caches the raw bytes to `/var/lib/cloud/instance/user-data.txt` (root, 0600) and executes nothing.

- `nsbootstrap.service` carries **no** `Condition*` **gating** (removed in ENG-1185239 / ENG-1188189 — a `Condition` skip leaves no log or journal entry, the very blind spot to eliminate). It always runs and reports what it found: absent / empty userdata and the already-applied case are logged and exit 0. Idempotency is enforced by an in-process `bootstrap.json.done` check, not a systemd condition.
- A **root-only** `ExecStartPre=+/usr/bin/install -m 0640 -o nsadmin ...` stages the root-owned cache into a restricted, nsadmin-readable copy — a fixed file copy, not arbitrary code. Secrets are never world-readable. The main process stays unprivileged (`User=nsadmin`).

7.4 Layer 3 — strict validation (fail closed)

- Parse strictly as JSON — a shell script or YAML `#cloud-config` is not JSON and is rejected.
- **Allowlist** top-level keys (`dlpaas`, `dns`, `interface`, `management-plane`, `system`, `request`, `persona`); any unknown key invalidates the whole file.
- Per-field type/enum validation; clib re-validates every command it receives.
- **No** `commands[]` — the raw-command escape hatch has been removed, so even the trusted processor has no arbitrary-command path.

7.5 Defense in depth

- **Run-once** — the done-marker + finalize prevent re-processing on later boots.
- **IMDSv2 required** on the instance (blocks SSRF reads of metadata/userdata).
- **IAM** restricts `ec2:RunInstances` / `ModifyInstanceAttribute` (`userData`) to the provisioning pipeline role — the upstream gate this hardening backstops.
- **Immutability** — with `write_files` gone, userdata cannot drop a systemd unit, cron entry, sudoers file, `authorized_keys`, or overwrite the nsbootstrap binary.
- **Auditability** — every applied verb is logged; the redacted `bootstrap.json.done` is a tamper-evident record.

Control	Effect

Strip exec/write/fetch modules + restrict datasources + disable vendor-data	cloud-init cannot run <code>runcmd /scripts/config-mgmt/ write_files</code>
Raw-JSON delivery out-of-band (no <code>write_files</code>) + root-only staged copy	no file-write vector; appliance immutable to userdata; least privilege preserved
Strict schema validation, closed verb vocabulary, no <code>commands[]</code>	only pre-authorized config is applied; hostile input fails closed
Run-once + IMDSv2 + IAM on userData	shrinks and gates the remaining surface

8. bootstrap.json Schema

Structured config only. A missing/empty section is skipped. `persona` is optional (`dlp-on-demand` or `dspm`). The keys below match the shipped `nsbootstrap` schema. **No** `commands[]` (removed).

```

1  {
2  {
3      "dlpaas": { "server-cert": "", "server-key": "", "server-intermediate-ca-chain": "" },
4      "dns": { "primary": "", "secondary": "" },
5      "interface": { "v4": { "dhcp": { "enable": false },
6                      "static": { "enable": true, "ip": "", "gw": "", "netmask": "" } } },
7      "management-plane": { "upstream-proxy-server":
8                          { "hostname": "", "port": "", "username": "", "password": "",
9                          "trusted-ca": "" } },
10     "system": { "hostname": "", "licensekey": "", "login-banner": "",
11               "metrics": { "debug": false }, "ssh-allowlist": [],
12               "ssh-public-keys": [ { "key": "", "user": "" } ], "timezone": "" },
13     "request": {
14         "self-signed": {
15             "days": "", "country": "", "state": "", "city": "",
16             "organization": "", "organization-unit": "", "common-name": "", "email-address": ""
17         },
18         "certificate-request": {
19             "days": "", "country": "", "state": "", "city": "",
20             "organization": "", "organization-unit": "", "common-name": "", "email-address": ""
21         }
22     },
23     "persona": ""
24 }

```

Field rules: `interface.v4` `static` and `dhcp` are mutually exclusive, and `static ip / gw / netmask` are required together. `dlpaas` requires all three of `server-cert / server-key / server-intermediate-ca-chain` together. Each `request` subsection requires all eight DN fields together; `self-signed` may be supplied alone or with `certificate-request`, but `certificate-request` (CSR) is **not** allowed on its own. Secrets (`system.licensekey`, `management-plane.upstream-proxy-server.password`, `dlpaas.server-key`) are redacted from `bootstrap.json.done`.

9. Superseded Design (for the record)

✓ Original in-wizard design — deprecated

The first design applied bootstrap config **inside the** `nsconfigwizard` **TUI**: the wizard read `/opt/ns/cfg/bootstrap.json` (written by cloud-init `write_files`), generated commands, applied them over `clib`, then applied the persona after `nssetupdlpaas` completed. It also supported a raw `commands[]` escape hatch.

Why it was replaced:

- Coupled a headless provisioning step to an interactive login shell.
- Fragile cloud-init ↔ `nssetupdlpaas` ordering; attempts to gate on `cloud-final.service` produced systemd **ordering cycles** (jobs deleted) and, when waiting on cloud-init in-process, a **boot deadlock** (`cloud-init status --wait`).
- `write_files` delivery is an arbitrary-file-write hole; the raw `commands[]` hatch is an arbitrary-command hole — both unacceptable for a security-sensitive appliance.

Resolved by: the standalone `nsbootstrap.service` (§2-4), `multi-user.target` anchoring (§3), cloud-init lockdown + raw-JSON delivery + strict validation (§7), and removing `commands[]`.

10. Deferred / Open Items

- `add system ssh-public-keys` is not idempotent on a re-run before finalize (duplicate keys appended).
- **Known issue**: a hyphen in the local-part of `request.*.email-address` is rejected by the `clib` email validator (e.g. `user-name@example.com`); workaround: use an email without a hyphen in the local-part.
- CloudFormation / Terraform / userdata structure (separate design) — must deliver bootstrap config as **raw JSON** userdata and enforce IMDSv2 + userData IAM.

11. Testing

- **nsbootstrap**: read/apply ordering, progress-callback secret-safety, atomic progress writer + schema, config-section builders, `clib` client, and **security validation** (unknown-key / shell-script / `#cloud-config` payloads rejected).
- **nsconfigwizard**: observer helpers, path-parameterized progress reader, persona-apply refactor.

```
1  
2 cd components/appliance/nsbootstrap    && tox -e py38  
3 cd components/appliance/nsconfigwizard && tox -e py38
```

